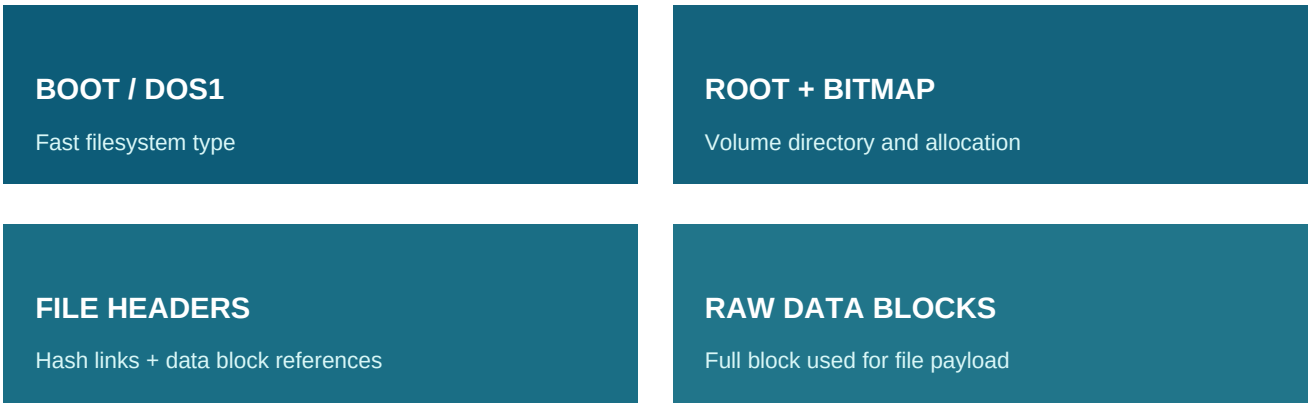
Amiga Fast File System - FFS

OFFICIAL FORMAT / CHRONOLOGY / ENGINEERING

The AmigaOS FFS format accelerated disk access by removing the per-data-block header required by the original OFS format.

This guide documents the original format's technical architecture, history and independent source references. InfiltratorOS modules and install packages are implementation artefacts, not the authority for the historic on-disk definition.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

Amiga Fast File System - FFS

HISTORICAL CHRONOLOGY

The first Amiga machines shipped in 1985 with the Original File System (OFS). When larger hard disks and applications required better sequential throughput, Amiga Fast File System was introduced in the late 1980s and became associated with AmigaOS 1.3-era hard disk use. It was a storage format change, not simply a faster driver reading exactly the same OFS file data blocks.

Historical AmigaDOS types identify Fast File System variants: DOS1 for original FFS, DOS3 for international case handling and DOS5 for directory-cache support. Later long-name extensions added further modes. The AmigaOS and Linux AFFS documents distinguish these on-media variations and support limitations; a reader must detect the actual DOS type before interpreting data.

Amiga Fast File System - FFS

ORIGINAL ON-DISK ARCHITECTURE

FFS preserves the Amiga model of boot blocks, root block, allocation bitmap, hashed directory structures, file headers and extension blocks. The decisive difference from OFS is that FFS ordinary file data blocks contain raw file payload rather than the 24-byte OFS data header. A data block can therefore carry more useful bytes and be accessed more efficiently.

File header blocks and extension blocks contain the table of data-block pointers. Because FFS does not retain OFS data-block headers containing reverse information, damaged file headers may make recovery of payload ownership more difficult. The root and metadata blocks still have structured fields and checksums, but ordinary FFS file payload blocks are not equivalent to checksummed OFS records.

Amiga Fast File System - FFS

ALLOCATION AND DIRECTORIES

The allocation bitmap records available blocks. Directories use hash-table slots and chains of file or directory header blocks, with Amiga metadata including names, protections, timestamps and file comments. File extension blocks expand the pointer capacity when a file exceeds what its header can describe. The storage layout must be interpreted with the correct block size and partition origin.

FFS reuses much of the Amiga namespace model but can differ between international, directory-cache and long-name types. Case folding, caching state and name limits are mode-dependent. Hard-link and symbolic-link representations are not automatically identical to the full semantics of a Unix VFS.

Amiga Fast File System - FFS

INTEGRITY AND RECOVERY

FFS does not have a general journal comparable to ext3 or NTFS. The root block can contain a bitmap-valid flag, and metadata must be checked before write access after an interrupted operation. File data no longer has the OFS header and checksum redundancy, so recovery relies more heavily on trustworthy file header and extension-block references.

A checker should validate directory hash chains, file headers, bitmap use, checksum fields, block ownership and reference completeness. Reconstruction from raw data is inherently more speculative than on OFS because ownership sequence numbers are not available in each data block. The ability to recover a partly overwritten file must not be exaggerated.

Amiga Fast File System - FFS

DESIGN RATIONALE AND LIMITATIONS

FFS trades some of OFS's per-block redundancy for greater throughput and data capacity per block, an appropriate choice for hard disks. The format remains bitmap-and-header based rather than a FAT or B-tree filesystem. Its layout must account for reserved blocks and the root block's geometry.

A modern implementation should support multiple legitimate block sizes, DOS types and Amiga partitioning conventions, including RDB. Neither an Amiga filesystem name nor a partition size is sufficient to identify data-block semantics. The FFS improvements did not add end-to-end checksums or a full modern transaction model.

Amiga Fast File System - FFS

INFILTRATOROS INTEGRATION BOUNDARY

FFS.so and OFS.so are distinct filesystem modules. Treating their data blocks interchangeably is unsafe, even where they share metadata code and AmigaDOS namespace concepts. Any defragmentation or repair algorithm must preserve mode-specific allocation and file-header relationships.

Our published .so and optional .ko are implementation products. Kernel AFFS documentation and Amiga developer references remain the authorities on actual DOS1/3/5 variants, not the appearance of a generic PDF diagram.

Amiga Fast File System - FFS

PRIMARY SOURCE REFERENCES

These primary technical documents and near-contemporary references provide provenance for the design history and on-disk structures. For exact field sizes or a particular format revision, consult the original specifications and source code, not the conceptual cover illustration.

1. Linux kernel: Overview of Amiga Filesystems

https://kernel.org/doc/html/v6.7/filesystems/affs.html

2. Amiga Developer filesystem evolution and long-name structures

https://wiki.amigaos.net/wiki/DCFS_and_LNFS_Low_Level_Data_Structures

3. ADFlib Amiga disk format technical reference

https://adflib.github.io/FAQ/adf_info.html

Our independent .so, optional .ko, build procedures and test evidence describe a separate implementation. The source-level history in this PDF cannot itself qualify production writes, boot, recovery or cross-platform compatibility.

FFS / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Verify the boot-block DOS identifier and the appropriate FFS variant before processing block headers, filenames and directory metadata.

2. Unlike OFS, FFS file data blocks are raw payload data without the 24-byte OFS header. The difference affects throughput and the amount of usable data per block.

3. Root and directory header blocks still contain hash tables and links for namespace lookup; FFS did not replace the directory system with a FAT.

4. Follow directory hash chains with checksums and type validation. File and extension header blocks carry the file's table of data-block references.

5. Interpret Amiga big-endian metadata, file timestamps, protection bits, comments and name lengths using the selected DOS variant's rules.

6. Block pointers are partition-relative. The root placement and bitmap positions cannot be inferred safely from a generic partition table without checking volume geometry.

7. Treat international and directory-cache format variants as real on-media differences, not merely mount-time performance switches.

8. Check logical file length independently of the number of referenced blocks, especially for partially filled final blocks and recovered file headers.

FFS / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. FFS removes per-data-block OFS checksums, so a damaged data payload may not be detectable through the metadata checksum chain.

2. Check root, directory and extension block checksums and confirm that all data-block addresses remain inside partition bounds.

3. Do not apply OFS data-block parsing to FFS file content; doing so would discard or misinterpret 24 bytes from every data block.

4. Preserve directory-cache and international-name semantics when editing, including the selected case-comparison rules and cache maintenance.

5. Compare actual rooted allocation with bitmap state before declaring an image repairable. Orphaned or multiply referenced blocks require explicit reporting.

6. Distinguish filesystem block addressing from RDB partition information and physical storage offsets; hard-disk geometry requires careful translation.

7. An interrupted namespace update can leave individually valid headers but inconsistent parent, extension or bitmap relationships.

8. Use original AmigaDOS specifications and kernel AFFS documentation to validate field layouts. An independent Linux adapter does not define the historical disk format.